

hw4: mmap() – eager and lazy

Due by noon, Wed Oct 11, 2023

In this homework, we learn a little bit about memory management by adding a widely used Unix feature to xv6: `mmap()`. While `mmap` can be (and is) used to allocate large chunks of memory, the more interesting use of `mmap()` is that of mapping a file into memory. This means, you call `mmap` once, with a file descriptor as argument, and the file immediately appears in virtual memory.

There are at least two main ways to do this. The first is **eager**: when `mmap` is called, have the kernel read the file as instructed, put it somewhere convenient in memory, and return a pointer to the location. The second is **lazy**: take note of the user's request, and return without doing anything further. When the user tries to access the memory, deal with the resulting page-fault, reading in only those pages that the user tries to read, when the user tries to read them.

Getting started

As usual, create your hw4 repository using this link: <https://classroom.github.com/a/DJr267dQ>

A solution template is provided in the public class repository, under the name `hw4`.

The changes on top of the master branch can be seen using [this link](#). (select the tab "files changed" to see the differences). Read the diff carefully for changes related to the homework.

The correct implementation will make three new programs-- `eager`, `lazy`, and `bad` --print expected results.

Read the three programs' source code to understand the expected behavior of them.

sys_mmap()

The system call `sys_mmap()` has already been created for you, only it doesn't do much yet. **The majority of the homework solution should go here, or in functions called by `sys_mmap()`**. When you run the test programs `eager` and `lazy`, they crash with the current `sys_mmap()` implementation. This is expected.

Part 1: Eager solution (60%)

In the eager solution, where the second argument of the `mmap` system call is 0, `sys_mmap()` allocates and maps all the memory needed for the file contents (see `allocuvmm()` for an example of how to do this. `allocuvmm` is only for the heap), and reads all of the file contents into the newly allocated memory. To read part of a file, you can use `fileread()` in `file.c`, **or** borrow some code in `fileread()` to directly call `readi()`. You only need the code that handles `type == FD_INODE`.

For both the eager and the lazy, you should map `mmap()` regions into addresses `0x400000000000` and above. You should expect there are multiple calls to `mmap()`. You should not overwrite or reuse any `mmap` areas that have been created by earlier calls. You'll need to store some information inside `struct proc`. You need to add a few lines of code for necessary initialization. See the TODO in `proc.c`.

A macro has been added to `memlayout.h`:

```
#define MMAPBASE 0x0000400000000000
```

To test your program, run the user program `eager`. It should show the output posted below.

Part 2: Lazy solution (40%)

The lazy solution works quite differently. When the `lazy` argument is 1, `sys_mmap()` should not allocate or map any memory, nor does it read anything from disk. Instead, it records the request in `struct proc`, and returns a pointer immediately. Initially, this pointer points to an unallocated and unmapped part of the virtual address space.

Later, when the process tries to read from or write to the memory area it just `mmap`-ed, a page fault occurs. You need to implement the page fault handling code (`handle_pagefault`) that allocates the appropriate page to serve this read/write, and fills the page with the appropriate contents from the file. To figure out what address the program was trying to access, we just need to read the CR2 register. The hook code is already provided for you in `trap.c`. See "case `T_PGFLT`:" for details.

To test your program, run the user program `lazy`. The hw4 template has a modification to `exit()`, that prints out the number of system free pages before exiting. **The lazy solution should have about 17 more free pages than the eager solution**. A full sample of the correct solution output is provided below.

Sample output

Note that you may see different absolute numbers.

The difference between the two numbers in your solution should be very close to 17.

```
$ eager
About to make first mmap. Next, you should see the first sentence from README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6)

Second mmap coming
xv6 @ UIC ROCKS!!!

Checking that first mmap is still ok, you should see the first sentence from README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6)
Exiting process. System free pages is 57003
$ lazy
About to make first mmap. Next, you should see the first sentence from README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6)

Second mmap coming
xv6 @ UIC ROCKS!!!

Checking that first mmap is still ok, you should see the first sentence from README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6)
Exiting process. System free pages is 57020
```

Notes on hw4: mmap

The very original motivation for the `mmap` system call is to map a file into the process' address space.

- It's intuitive to use read/write system calls to transfer file contents: user <-> kernel <-> disk
- `mmap` provides a clean interface and rich functionality.

`mmap()` in Linux

- On-demand file I/O handled transparently by the OS (through page-fault exception handler).
- Sync a range of mapped file back to the disk -- you specify the memory address, kernel figures out where to write on the disk.
- Allocating process memory without a backing file. It's also used for allocating "huge pages".

Back to xv6:

- `eager mmap`: allocate process memory and read file content into it -- page faults are not involved. In other words, everything should be done in the `mmap` system call.
- `lazy mmap`: record the metadata for the process about what file is mapped to what address range. When the syscall returns, pages are still unallocated and the first access to each virtual page will raise a page faults (traps.h: `T_PGFLT`). Using the metadata associated with the process, the page fault handler can allocate memory and load the corresponding file contents one page at a time.

The context of the `mmap` system call (`mmap/sys_mmap`)

- Current process (the per-code global variable `proc`)
- An active file descriptor/struct file (the first syscall argument)
- A flag (0/1) to choose eager or lazy mapping. (the second syscall argument)

The procedure of eager `mmap`:

- First, we need to know the size of the mapping, which should be equal to the file size. In hw4, we only consider regular files, which corresponds to the "inode" with type `FD_INODE`.
- Record/update the metadata for the process.
- Allocate the memory for the user process. The xv6 kernel already provided a few functions for memory-related operations. See Hints below.
- Read the entire file's data into the newly allocated memory. You may use either the kernel address or the user address. Make sure the addresses used are "really" valid when being accessed.

The procedure of lazy `mmap`:

- Only the first two steps in the eager `mmap` should be done in the syscall, as we are being LAZY!

To make lazy `mmap` to work correctly, we need to install the page fault handler in the kernel, the corresponding trap number is `T_PGFLT`, defined in `traps.h`. Check the added the code in `trap.c` and think: what causes the execution to reach this place?

When handling a page fault, the faulty address is stored in `%cr2` by the CPU. `rcr2()` function read the `%cr2` register.

More hints:

- `mappages()` are already provided. read the code first to see how can it help you.
- `kalloc()` -- allocate physical pages
- It's recommended to finish each file's mappings at an aligned address. For example, mmap of a 5KB file will occupy two pages from `MMAPBASE` to `MMAPBASE + 8KB`. The next mmap will start from `MMAPBASE + 8KB`, leaving an accessible but uninitialized gap between the two memory ranges. The `PGROUNDUP` macro in `mmu.h` can be helpful.**
- The CPU never check the updates in the page table. We need to flush the TLB to force the CPU to use the update-to-date mappings. reloading the page table root into the system register `%cr3` does this. `switchkm()` shows an example.
- Read `filewrite()` and `fileread()` in `file.c`